



Red Hat OpenShift AI Self-Managed 3.5

Getting started with Red Hat OpenShift AI Self-Managed

Learn how to work in an OpenShift AI environment

Red Hat OpenShift AI Self-Managed 3.5 Getting started with Red Hat OpenShift AI Self-Managed

Learn how to work in an OpenShift AI environment

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Learn how to work in an OpenShift AI environment.

Table of Contents

CHAPTER 1. OVERVIEW	3
1.1. DATA SCIENCE WORKFLOW	3
1.2. ABOUT THIS GUIDE	4
1.3. GLOSSARY OF COMMON TERMS	4
CHAPTER 2. LOG IN TO OPENSIFT AI	14
2.1. VIEW INSTALLED OPENSIFT AI COMPONENTS	14
CHAPTER 3. CREATE A PROJECT	16
CHAPTER 4. CREATE A WORKBENCH AND SELECT AN IDE	17
4.1. WORKBENCH IMAGES	17
4.1.1. Administrator control over image visibility	18
4.2. CREATE A WORKBENCH	18
CHAPTER 5. NEXT STEPS	23
5.1. ADDITIONAL RESOURCES	24

CHAPTER 1. OVERVIEW

Red Hat OpenShift AI is an artificial intelligence (AI) platform that provides tools to rapidly train, serve, and monitor machine learning (ML) models onsite, in the public cloud, or at the edge.

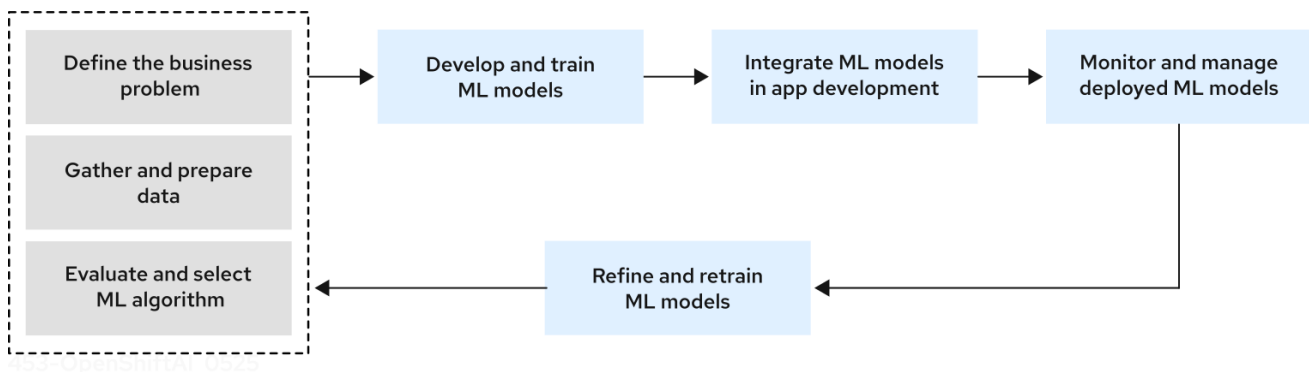
OpenShift AI provides a powerful AI/ML platform for building AI-enabled applications. Data scientists and MLOps engineers can collaborate to move from experiment to production in a consistent environment quickly.

You can deploy OpenShift AI on any supported version of OpenShift, whether on-premise, in the cloud, or in disconnected environments. For details on supported versions, see [Supported Configurations for 3.x](#).

1.1. DATA SCIENCE WORKFLOW

For the purpose of getting you started with OpenShift AI, the following figure illustrates a simplified data science workflow. The real world process of developing ML models is an iterative one.

Figure 1.1. Simplified data science workflow



The simplified data science workflow for predictive AI use cases includes the following tasks:

- Defining your business problem and setting goals to solve it.
- Gathering, cleaning, and preparing data. Data often has to be federated from a range of sources, and exploring and understanding data plays a key role in the success of a project.
- Evaluating and selecting ML models for your business use case.
- Train models for your business use case by tuning model parameters based on your set of training data. In practice, data scientists train a range of models, and compare performance while considering tradeoffs such as time and memory constraints.
- Integrate models into an application, including deployment and testing. After model training, the next step of the workflow is production. Data scientists are often responsible for putting the model in production and making it accessible so that a developer can integrate the model into an application.
- Monitor and manage deployed models. Depending on the organization, data scientists, data engineers, or ML engineers must monitor the performance of models in production, tracking prediction and performance metrics.
- Refine and retrain models. Data scientists can evaluate model performance results and refine models to improve outcome by excluding or including features, changing the training data, and modifying other configuration parameters.

1.2. ABOUT THIS GUIDE

This guide assumes you are familiar with data science and ML Ops concepts. It describes the following tasks to get you started with using OpenShift AI:

- Log in to the OpenShift AI dashboard
- Create a project
- If you have data stored in Object Storage, configure a connection to more easily access it
- Create a workbench and choose an IDE, such as JupyterLab or code-server, for your data scientist development work
- Learn where to get information about the next steps:
 - Developing and training a model
 - Automating the workflow with pipelines
 - Implementing distributed workloads
 - Testing your model
 - Deploying your model
 - Monitoring and managing your model

See also [OpenShift AI tutorial: Fraud detection example](#) . It provides step-by-step guidance for using OpenShift AI to develop and train an example model in JupyterLab, deploy the model, and refine the model by using automated pipelines.

1.3. GLOSSARY OF COMMON TERMS

This glossary defines common terms for Red Hat OpenShift AI.

accelerator

A specialized hardware component, such as a GPU, TPU, or specialized AI accelerator (for example, Intel Gaudi, AMD Instinct), designed to offload compute-intensive tasks from the CPU to increase processing efficiency. In OpenShift AI, administrators use accelerator profiles to configure and manage user access to these hardware resources.

Agent Card

A machine-readable metadata document (typically in JSON format) that describes an AI agent's identity, capabilities, supported protocols, and authentication requirements. Agent Cards enable dynamic agent discovery and are a core component of the A2A protocol.

agent identity

The set of credentials, roles, and permissions assigned to an AI agent, distinct from the identity of the user who invoked it. Agent identity enables fine-grained access control, auditability, and least-privilege enforcement for autonomous agent actions.

agent operations (AgentOps)

A framework for monitoring and managing AI agents in real time, providing visibility into agent decision-making processes, identity-based access control, safety guardrails, and lifecycle management for autonomous agent workflows.

agent registry

A centralized catalog where teams can publish, discover, browse, and govern deployed AI agents. An agent registry provides visibility into what agents exist, what they do, and who owns them, preventing duplication and enabling reuse across an organization.

agent sandbox

An isolated execution environment in which an AI agent runs with restricted access to system resources, network, and tools. Sandboxing contains the blast radius of unintended agent actions and enforces security boundaries between agent workloads and the host system.

Agent-to-Agent protocol (A2A)

An open protocol that enables AI agents to communicate, discover each other's capabilities, and collaborate on tasks across different frameworks and vendors. A2A uses Agent Cards for capability advertisement and supports secure, asynchronous multi-agent workflows.

Agentic AI

AI systems capable of independent reasoning, planning, and tool execution to carry out complex tasks with limited supervision.

agentic loop

The iterative cycle in which an AI agent perceives its environment, reasons about the next action, executes that action (often via tool calls), observes the result, and repeats until the task is complete or a termination condition is met.

AI agent (Autonomous Agent)

A software system that uses AI to perceive its environment, make decisions, and take actions to achieve specific goals with minimal human intervention.

AI Gateway

A centralized service that provides a unified interface for accessing multiple AI models and services, handling request routing, authentication, rate limiting, and monitoring.

AI pipelines

A workflow engine used to define, schedule, and automate repeatable sequences of tasks, such as data preprocessing, model training, and evaluation.

artificial intelligence (AI)

The capability to acquire, process, create, and apply knowledge to make predictions, recommendations, or decisions.

AutoRAG

A framework for constructing and optimizing retrieval-augmented generation (RAG) pipelines, automating the configuration and tuning of RAG components to improve retrieval accuracy and generation quality.

bias detection

The process of calculating fairness metrics to identify when an AI model produces unfair or discriminatory outcomes based on sensitive attributes.

chunking

The process of splitting documents into smaller, semantically meaningful segments for ingestion into a vector database. Chunk size and overlap strategy directly affect retrieval quality in RAG pipelines.

Confidential Containers (CoCo)

A Kubernetes-native framework that runs container workloads inside hardware-backed Trusted Execution Environments (TEEs), providing memory encryption and attestation. Used for AI workloads that require protection of model weights or sensitive data from infrastructure operators.

connection type

The type of external source to connect to from a project, such as an OCI-compliant container registry, S3-compatible object storage, or Uniform Resource Identifiers (URIs).

context window

The maximum number of tokens a language model can process in a single inference call, encompassing both the input prompt and the generated output. Context window size determines how much information (retrieved documents, conversation history, tool results) an agent can reason over at once.

continuous batching

A serving optimization technique that dynamically groups individual inference requests into batches as they arrive, improving throughput and GPU utilization without requiring clients to wait for a full batch to form before processing begins.

custom resource (CR)

An extension of the Kubernetes API that creates a specific instance of a Custom Resource Definition (CRD). Custom resources allow users to introduce custom objects into an OpenShift cluster that behave like native Kubernetes objects (such as pods or services).

custom resource definition (CRD)

A Kubernetes API resource that defines a new, unique object type (Kind) within an OpenShift cluster. Creating a CRD allows the Kubernetes API server to manage the lifecycle of custom resources associated with that definition.

data connection

A configuration that stores the parameters required to connect to an S3-compatible object storage, database or OCI-compliant container registry from a project.

data science project

A dedicated workspace used to organize and isolate all resources related to a data science initiative, including workbenches, connections, and pipelines.

Direct Preference Optimization (DPO)

A model alignment technique that trains language models directly on preference pairs (preferred vs. rejected responses) without requiring a separate reward model, offering a simpler alternative to RLHF for fine-tuning model behavior.

disaggregated compute-cluster scaling

An architecture pattern that separates compute resources from storage and allows independent scaling of different components in a distributed system, optimizing resource utilization and cost efficiency.

disconnected data federation

A capability that enables querying and combining data from multiple sources in a disconnected or air-gapped environment without requiring direct network connectivity to external data stores.

disconnected environment

A deployment environment on a restricted or air-gapped network without an active internet connection.

distributed orchestration stack (CodeFlare, Ray, Kueue)

A collection of frameworks that coordinate and manage distributed computing tasks across multiple nodes, including CodeFlare for simplified distributed training, Ray for general-purpose distributed computing, and Kueue for batch job queueing.

distributed workloads

Workbench, machine learning or data processing, or model serving workloads that are partitioned and executed simultaneously across multiple nodes or GPUs within an OpenShift AI cluster.

Docling

A document processing tool that converts unstructured documents into machine-readable data at scale while preserving lineage and traceability from source documents to processed outputs.

drift detection

The process of monitoring a deployed model's predictions or behavior over time to identify degradation caused by changes in input data distributions (data drift) or shifts in the relationship between inputs and outputs (concept drift).

embedding

A dense numerical representation of text or other data, expressed as a vector of floating-point numbers. Embedding models, which are neural networks, convert input data into embeddings that capture semantic meaning, enabling similarity-based retrieval. In Red Hat OpenShift AI, embeddings are generated by a configured embedding model and stored in a vector database so that the most relevant content can be retrieved in response to a query.

EvalHub

A unified interface and API for assessing, benchmarking, and certifying AI models, providing immutable assets that create a verifiable, auditable record of model performance and evaluation results.

fine-tuning

The process of adapting a pre-trained foundation model to a specific task, for example either through full fine-tuning or parameter-efficient fine-tuning (PEFT).

foundation model

A large AI model trained on broad, general-purpose data that can be adapted to a variety of downstream tasks.

function/tool calling

A capability that enables language models to invoke external functions or APIs to perform specific tasks, such as retrieving information, executing calculations, or interacting with external systems.

generative AI operations (GenAIOps)

Operational practices and tooling specifically designed for managing generative AI systems, including monitoring, governance, and lifecycle management of large language models and generative AI applications.

graphics processing unit (GPU)

A specialized hardware accelerator designed with a parallel architecture to rapidly manipulate memory and process data simultaneously. While originally designed for rendering graphics, GPUs are heavily utilized in machine learning for acceleration during model training, fine-tuning, and inference workloads.

grounding

The practice of anchoring a language model's outputs to verifiable data sources, reducing hallucinations by constraining generation to facts retrieved from documents, databases, or APIs rather than relying solely on the model's parametric knowledge.

guardrails

A set of safety, security, and compliance mechanisms integrated into the AI lifecycle or inference pipeline to evaluate, validate, and sanitize user prompts and model outputs. Guardrails ensure that generative AI models operate within predefined operational, ethical, and corporate boundaries, helping to mitigate risks such as toxic content, hallucinations, jailbreaking, and accidental data leakage.

GuideLLM

A tool for evaluating large language model performance and quality metrics.

hallucination

An output from a language model that is fluent and plausible but factually incorrect, fabricated, or unsupported by the provided context. Hallucinations are a primary risk in production AI systems, mitigated through techniques such as RAG, grounding, and guardrails.

hardware profile

A configuration in OpenShift AI that specifies the type and quantity of hardware accelerators to be allocated to a workbench, pipeline, or model server.

hardware-agnostic runtime execution

The capability to run AI workloads on different hardware platforms without requiring code changes, abstracting the underlying compute infrastructure.

hybrid cloud foundation

An infrastructure architecture that seamlessly integrates on-premises data centers with public and private cloud resources, enabling workload portability and unified management.

hybrid-cloud data persistence

Storage solutions that maintain data consistency and availability across on-premises and cloud environments, supporting data mobility and resilience.

inference

The real-time or batch process of executing data through a deployed, trained machine learning model to generate a prediction, classification, or token generation output. In production systems, inference is enabled through model serving platforms like KServe and ModelMesh.

inference server

The underlying software engine or container process (such as vLLM, Caikit, or Triton Inference Server) that hosts a model, handles high-throughput API routing, manages memory optimizations like KV caching, and executes the mathematical transformations required for inference.

jailbreak

An adversarial technique that manipulates a language model into bypassing its safety instructions and behavioral constraints, typically through crafted prompts that exploit the model's instruction-following behavior to produce prohibited or harmful outputs.

Kata Containers

A container runtime that launches each container inside a lightweight virtual machine, providing hardware-enforced isolation between workloads. In OpenShift AI, Kata Containers (via OpenShift Sandboxed Containers) provide VM-level isolation for high-security agent workloads.

KServe

A Kubernetes-native model serving framework providing scalable, standards-based inference.

KV cache

A memory structure that stores previously computed key-value pairs from a transformer model's attention layers during autoregressive generation. Reusing cached KV pairs avoids redundant computation for earlier tokens, significantly accelerating inference. Techniques like PagedAttention and prefix caching optimize KV cache memory management.

Large Language Model Distributed (LLM-D)

An open-source orchestration and scheduling framework designed for Kubernetes to optimize, scale, and manage large language model inference across high-performance cluster infrastructure. Integrated into cloud-native AI stacks like OpenShift AI, LLM-D optimizes hardware utilization by disaggregating compute-heavy prefill phases from memory-bandwidth-heavy decode phases and implementing prefix-cache-aware routing to minimize time-to-first-token (TTFT).

large language model (LLM)

A type of generative foundation model trained on vast amounts of text data, featuring billions of parameters, designed to understand, process, and generate human-like natural language or code.

Low-Rank Adaptation (LoRA)

A parameter-efficient fine-tuning technique that freezes the pre-trained model weights and injects trainable low-rank matrices into each transformer layer, enabling task-specific adaptation with a fraction of the memory and compute required for full fine-tuning.

machine learning (ML)

A subfield of artificial intelligence focused on developing algorithms that enable computers to learn patterns from data without explicit programming.

MCP Gateway

A centralized proxy that manages access to multiple MCP servers, providing authentication, authorization, rate limiting, usage tracking, and discovery of available tools. The gateway enables platform-level governance over which agents can access which tools under what conditions.

MCP server

A service that implements the Model Context Protocol server specification, exposing tools, resources, or prompts that AI agents can invoke. Each MCP server provides a well-defined interface to a specific capability (database access, API integration, file operations) that agents discover and call at runtime.

MLflow

An open-source platform for managing the end-to-end machine learning lifecycle, including experiment tracking, model versioning, deployment, and collaboration.

MLOps (machine learning operations)

A set of practices and cultural philosophies that unifies machine learning system development (Dev) and system operations (Ops). MLOps standardizes and automates the continuous integration, continuous delivery (CI/CD), deployment, monitoring, and governance of machine learning lifecycles in production.

MLServer

An inference server designed for production machine learning workloads that supports multiple frameworks and provides features such as adaptive batching and metrics collection.

model

In a machine learning context, a set of functions and algorithms that have been trained and tested on a data set to provide predictions or decisions.

Model-as-a-Service (MaaS)

A deployment model that provides access to pre-trained AI models through APIs, enabling users to consume model capabilities without managing the underlying infrastructure.

model catalog

A curated, user-facing directory within OpenShift AI where data scientists can discover, evaluate, and select verified generative AI and foundation models appropriate for their business use cases.

Model Context Protocol (MCP)

A protocol that creates a standardized communication channel between AI agents and enterprise data sources, enabling agents to access live data, APIs, and tools with identity-based access control and built-in guardrails.

model registry

A centralized repository and metadata service used to manage the lifecycle of machine learning models. It tracks model lineages, versions, training hyperparameters, performance metrics, and deployment states from development through to production.

model server

An operational component in OpenShift AI (configured either as a single-model platform via KServe or a multi-model platform via ModelMesh) that wraps an inference server engine inside a Kubernetes pod to expose network endpoints for serving models.

model serving

The practice of making a trained model accessible as a networked service to enable inference. Real-world applications send inference requests to the service using a REST or gRPC API and receive predictions.

model-serving runtime

The designated software environment or framework configurations used to prepare and optimize a specific model format (such as ONNX, TensorFlow, or Hugging Face weights) for inference deployment.

ModelMesh

A multi-model serving framework optimized for high-density, cost-efficient deployment of many models.

multi-agent orchestration

A framework for coordinating multiple AI agents to collaborate on complex tasks, managing their interactions, workflows, and shared context.

multi-model serving platform (ModelMesh)

A serving architecture that efficiently hosts multiple models on shared infrastructure, providing dynamic model loading, intelligent placement, and resource optimization.

neural processing unit (NPU)

A specialized processor designed specifically for accelerating neural network operations and AI computations, optimized for matrix operations and tensor processing.

notebook

An interactive document that contains executable code, descriptive text, visualizations, and computational outputs used for data science experimentation and analysis.

notebook image

A container image that includes preinstalled tools, libraries, and an integrated development environment for developing machine learning models in a notebook interface.

notebook interface

An interactive document that contains executable code, descriptive text for that code, and the results of any code that is run.

object storage

A method of storing data, typically used in the cloud, in which data is stored as discrete units, or objects, in a storage pool or repository that does not use a file hierarchy but that stores all objects at the same level.

OGX (formerly known as Llama Stack)

A unified AI runtime environment that simplifies the deployment and management of generative AI workloads in Red Hat OpenShift AI. OGX integrates model inference, embedding generation, vector storage, and retrieval services into a single stack that is optimized for retrieval-augmented generation (RAG) and agent-based AI workflows. In Red Hat OpenShift AI, the OGX Operator manages the deployment lifecycle of these components.

OGXServer

A custom resource that declares the runtime configuration for an OGX server, including model providers, embedding configuration, vector storage, and persistence settings. When you create an OGXServer instance, the OGX Operator deploys and manages the corresponding OGX server in your Red Hat OpenShift AI project.

OpenShell

An agent-aware sandboxing platform that provides policy-controlled, isolated execution environments for AI agents. OpenShell enforces fine-grained restrictions on system calls, network access, and tool availability.

OpenShift Container Platform cluster

A deployment of OpenShift Container Platform consisting of control plane and worker nodes that provides the underlying infrastructure required to run OpenShift AI and its workloads.

OpenTelemetry (OTel)

A vendor-neutral observability framework for generating, collecting, and exporting telemetry data (traces, metrics, logs). In agentic AI systems, OpenTelemetry provides end-to-end visibility into agent decision chains, tool invocations, and latency across multi-step workflows.

operator (OpenShift Operator)

A method of packaging, deploying, and managing a Kubernetes application that extends the Kubernetes API to create, configure, and manage complex stateful applications on behalf of a Kubernetes user.

PagedAttention

A memory optimization algorithm used by vLLM that improves GPU utilization during inference by efficiently managing attention key-value caches in paged memory blocks.

persistent storage

A persistent volume that retains files, models or other artifacts across components such as model deployments, AI pipelines and workbenches.

persistent volume claim (PVC)

A request for storage in the cluster by a user.

pipeline

A repeatable sequence of automated steps for machine learning workflows.

prefix caching

An inference optimization that detects when multiple requests share the same prompt prefix (system prompt, few-shot examples) and reuses the computed KV cache for that prefix, reducing redundant computation and improving time to first token.

project

An organizational container for all data science resources in OpenShift AI, scoped to a Kubernetes namespace.

prompt injection

An attack in which adversarial content embedded in user input, retrieved documents, or tool outputs causes a language model to override its system instructions, potentially executing unauthorized actions or leaking sensitive information.

quantization

A model optimization technique that converts model weights and activations from high-precision data types (such as 32-bit floating-point) to lower-precision formats (such as 8-bit or 4-bit integers), reducing memory consumption and accelerating inference with minimal accuracy loss.

red teaming

The practice of systematically probing AI systems with adversarial inputs to discover safety vulnerabilities, bias, harmful outputs, or exploitable behaviors before deployment. Red teaming exercises may be manual (human testers) or automated (model-driven attack generation).

Reinforcement Learning from Human Feedback (RLHF)

A training technique that aligns language model outputs with human preferences by training a reward model on human rankings of model responses, then using reinforcement learning to optimize the language model against that reward signal.

reranking

A stage in a RAG pipeline where a secondary model rescores and reorders the initially retrieved document chunks by relevance to the query, improving precision before the final set of chunks is passed to the language model for generation.

retrieval-augmented generation (RAG)

A technique for enhancing large language models (LLMs) by integrating domain-specific data sources into the model's context at inference time. A RAG pipeline indexes content, builds an embedding store, retrieves the most relevant data in response to a query, and passes that data to the LLM alongside the user prompt. RAG enables models to produce accurate, verifiable answers grounded in data that was not included in the model's original training.

Runtime Catalog

A repository of available model-serving runtimes that defines the execution environments and frameworks supported for deploying machine learning models.

serving

The process of hosting a trained machine learning model as a network-accessible service. Real-world applications can send inference requests to the service by using a REST or gRPC API and receive predictions.

ServingRuntime

A custom resource definition (CRD) that defines the templates for pods that can serve one or more particular model formats. Each ServingRuntime defines key information such as the container image, supported model formats, and runtime configuration settings.

single-model serving platform (KServe)

A model serving architecture optimized for deploying individual models with dedicated resources, providing autoscaling and serverless capabilities.

Skill

A reusable capability or tool that can be invoked by AI agents to perform specific tasks, such as searching, computation, or integration with external systems.

Sovereign AI

A compliance-aware AI infrastructure approach that provides clear data lineage, governance controls, and regulatory compliance features, enabling organizations to maintain sovereignty over their AI systems and data.

SPIFFE/SPIRE

SPIFFE (Secure Production Identity Framework for Everyone) is a standard for issuing cryptographic workload identities. SPIRE is its production-grade runtime implementation. Together they provide platform-agnostic identity for workloads and agents, enabling mutual TLS and fine-grained authorization without static credentials.

starter kit

A pre-built, opinionated agent template that bundles a framework configuration, tool integrations, and deployment manifests for a common use case (such as a customer support agent or code assistant), enabling developers to bootstrap a working agent quickly.

Synthetic Data Generation Hub (SDG Hub)

A framework for constructing synthetic data generation pipelines that produce high-quality, domain-specific datasets with validation and auditability for model training and fine-tuning.

Time to First Token (TTFT)

A latency metric measuring the elapsed time from when an inference request is submitted to when the first output token is generated. TTFT is critical for interactive agent workloads where perceived responsiveness depends on how quickly the model begins responding.

token exchange

An OAuth 2.0 mechanism (RFC 8693) in which a user's credential is exchanged for a new, scoped token that an AI agent uses to access downstream services on behalf of that user. Token exchange enables least-privilege delegation without sharing the original credential.

tokenization

The process of breaking down text into smaller units called tokens, which serve as the basic units of processing for language models.

Training Hub

An algorithm-focused interface for large language model training that supports continual learning and reinforcement learning techniques while providing governance over datasets, training inputs, and model updates.

vector database

A database that stores data as dense numerical vectors, called embeddings, and supports efficient similarity search across those vectors. Vector databases are a core component of retrieval-augmented generation (RAG) pipelines, providing the retrieval layer that matches user queries to relevant stored content. In Red Hat OpenShift AI, supported vector databases include Milvus, Qdrant, and PostgreSQL.

vLLM

A high-throughput inference engine designed for running large language models. In Red Hat OpenShift AI, vLLM serves as the foundational engine for several single-model serving runtimes, optimizing LLM inference across GPUs, AI accelerators, CPUs, and other supported hardware.

workbench

An isolated environment for development and experimentation with ML models. Workbenches typically contain integrated development environments (IDEs), such as JupyterLab, RStudio, and Visual Studio Code.

workbench image

An image that includes preinstalled tools and libraries that you need for model development. Includes an IDE for developing your machine learning (ML) models.

YAML (YAML Ain't Markup Language)

A human-readable data serialization language used for Kubernetes configurations.

CHAPTER 2. LOG IN TO OPENSIFT AI

After you install OpenShift AI, log in to the OpenShift AI dashboard so that you can set up your development and deployment environment.

Prerequisites

- OpenShift AI is installed on your OpenShift cluster. For information, see [Installing and deploying OpenShift AI](#).
- You know the OpenShift AI identity provider and your login credentials.
 - If you are a data scientist, data engineer, or ML engineer, your administrator must provide you with the OpenShift AI instance URL, for example:

```
rh-ai.apps.example.abc.p1.openshiftapps.com
```

- You have the latest version of one of the following supported browsers:
 - Google Chrome
 - Mozilla Firefox
 - Safari

Procedure

1. Browse to the OpenShift AI instance URL and click **Log in with OpenShift**
 - If you have access to OpenShift, you can browse to the OpenShift web console and click the **Application Launcher** () → **Red Hat OpenShift AI**.
2. Click the name of your identity provider, for example, **GitHub**, **Google**, or your company's single sign-on method.
3. Enter your credentials and click **Log in** (or equivalent for your identity provider).

Verification

- The OpenShift AI dashboard opens on the **Home** page.

2.1. VIEW INSTALLED OPENSIFT AI COMPONENTS

In the Red Hat OpenShift AI dashboard, you can view a list of the installed OpenShift AI components, their corresponding source (upstream) components, and the versions of the installed components.

Prerequisites

- OpenShift AI is installed in your OpenShift cluster.

Procedure

1. Log in to the OpenShift AI dashboard.

2. In the top navigation bar, click the help icon () and then select **About**.

Verification

The **About** page shows a list of the installed OpenShift AI components along with their corresponding upstream components and upstream component versions.

Additional resources

- [Installing and managing Red Hat OpenShift AI components](#)

CHAPTER 3. CREATE A PROJECT

To implement a data science workflow, you must create a project. In OpenShift, a project is a Kubernetes namespace with additional annotations, and is the main way that you can manage user access to resources. A project organizes your data science work in one place and also allows you to collaborate with other developers and data scientists in your organization.

Within a project, you can add the following functionality:

- Connections so that you can access data without having to hardcode information like endpoints or credentials.
- Workbenches for working with and processing data, and for developing models.
- Deployed models so that you can test them and then integrate them into intelligent applications. Deploying a model makes it available as a service that you can access by using an API.
- Pipelines for automating your ML workflow.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have the appropriate roles and permissions to create projects.

Procedure

1. From the OpenShift AI dashboard, select **Projects**.
The **Projects** page shows a list of projects that you can access. For each user-requested project in the list, the **Name** column shows the project display name, the user who requested the project, and the project description.
2. Click **Create project**.
3. In the **Create project** dialog, update the **Name** field to enter a unique display name for your project.
4. Optional: If you want to change the default resource name for your project, click **Edit resource name**.
The resource name is what your resource is labeled in OpenShift. Valid characters include lowercase letters, numbers, and hyphens (-). The resource name cannot exceed 30 characters, and it must start with a letter and end with a letter or number.

Note: You cannot change the resource name after the project is created. You can edit only the display name and the description.
5. Optional: In the **Description** field, provide a project description.
6. Click **Create**.

Verification

- A project details page opens. From this page, you can add connections, create workbenches, configure pipelines, and deploy models.

CHAPTER 4. CREATE A WORKBENCH AND SELECT AN IDE

A workbench is an isolated area where you can examine and work with ML models. You can also work with data and run programs, for example to prepare and clean data. While a workbench is not required if, for example, you only want to service an existing model, one is needed for most data science workflow tasks, such as writing code to process data or training a model.

When you create a workbench, you specify an image (an IDE, packages, and other dependencies). Supported IDEs include JupyterLab, code-server, and RStudio (Technology Preview).

The IDEs are based on a server-client architecture. Each IDE provides a server that runs in a container on the OpenShift cluster, while the user interface (the client) is displayed in your web browser. For example, the Jupyter workbench runs in a container on the Red Hat OpenShift cluster. The client is the JupyterLab interface that opens in your web browser on your local computer. All of the commands that you enter in JupyterLab are executed by the workbench. Similarly, other IDEs like code-server or RStudio Server provide a server that runs in a container on the OpenShift cluster, while the user interface is displayed in your web browser. This architecture allows you to interact through your local computer in a browser environment, while all processing occurs on the cluster. The cluster provides the benefits of larger available resources and security because the data being processed never leaves the cluster.

In a workbench, you can also configure connections (to access external data for training models and to save models so that you can deploy them) and cluster storage (for persisting data). Workbenches within the same project can share models and data through object storage with the AI pipelines and model servers.

For projects that require data retention, you can add container storage to the workbench you are creating.

Within a project, you can create multiple workbenches. When to create a new workbench depends on considerations, such as the following:

- The workbench configuration (for example, CPU, RAM, or IDE). If you want to avoid editing the configuration of an existing workbench's configuration to accommodate a new task, you can create a new workbench instead.
- Separation of tasks or activities. For example, you might want to use one workbench for your Large Language Models (LLM) experimentation activities, another workbench dedicated to a demo, and another workbench for testing.

4.1. WORKBENCH IMAGES

A workbench image is preinstalled with tools and libraries for model development. You can use the provided images, or an OpenShift AI administrator can create custom images tailored to your needs.

Supported workbench images include the LLM compressor library for optimizing large language models for inference. For more information, see *Supported model compression workflows*.

To provide a consistent, stable platform for your model development, many provided workbench images contain the same version of Python. Most workbench images available on OpenShift AI are pre-built and ready for you to use immediately after OpenShift AI is installed or upgraded.

For information about Red Hat support of workbench images and packages, see [Supported Configurations for 3.x](#).

4.1.1. Administrator control over image visibility

OpenShift AI includes two types of workbench images:

Pre-installed images

Workbench images that are shipped with Red Hat OpenShift AI and managed by the operator. Pre-installed images are displayed on the **Workbench images** admin page with a **Pre-installed** label badge. These images are read-only: you cannot edit or delete them, but you can control their visibility.

Custom images

Workbench images imported by administrators to address specific project requirements. Custom images can be imported, edited, enabled, disabled, or deleted.

By default, all pre-installed images are enabled and visible to data scientists when they create workbenches. An administrator can disable individual pre-installed images from the **Workbench images** admin page. Disabled images do not appear in the image selection list when users create workbenches.

Disabling an image does not affect workbenches that were previously created with that image. Those workbenches continue to display the image name and version in the workbench details and edit views.

Visibility settings are stored as the **opendatahub.io/notebook-image-hidden** annotation on **ImageStream** resources. This annotation persists across OpenShift AI operator upgrades because it is applied by the administrator and is not included in the operator's shipped manifests.

A confirmation warning is displayed if you try to disable the last remaining enabled workbench image. Disabling all images prevents users from creating new workbenches until an image is re-enabled.

Additional resources

- [Hiding and showing pre-installed workbench images](#)
- [Supported model compression workflows](#)
- [Supported Configurations for 3.x](#)
- [Creating custom workbench images](#)

4.2. CREATE A WORKBENCH

When you create a workbench, you specify an image (an IDE, packages, and other dependencies). You can also configure connections, cluster storage, and add container storage.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a project.
- If you created a Simple Storage Service (S3) account outside of Red Hat OpenShift AI and want to connect to your existing S3 storage buckets, you have the following credential information:
 - Endpoint URL
 - Access key

- Secret key
- Region
- Bucket name

For more information, see [Working with data in an S3-compatible object store](#).

Procedure

1. From the OpenShift AI dashboard, click **Projects**, click the name of the project that you want to add the workbench to, and then click the **Workbenches** tab.
2. Click **Create workbench**.
The **Create workbench** page opens.
3. In the **Name** field, enter a unique name for your workbench.
Optional: Click **Edit resource name** to change the default resource name for your workbench. The resource name is used to identify your resource in Kubernetes. Valid characters include lowercase letters, numbers, and hyphens (-). The resource name cannot exceed 250 characters, and it must start with a letter and end with a letter or number.



NOTE

You cannot change the resource name after the workbench is created. You can edit only the display name and the description.

Optional: In the **Description** field, enter a description for your workbench.

4. In the **Workbench image** section, complete the fields to specify the workbench image to use with your workbench.
From the **Image selection** list, select a workbench image that suits your use case. A workbench image includes an IDE and Python packages (reusable code). If project-scoped images exist, the **Image selection** list includes subheadings to distinguish between global images and project-scoped images.



NOTE

The **Image selection** list shows only workbench images that have been enabled by an administrator. If an expected workbench image is not visible, contact your OpenShift AI administrator.

Optional: Click **View package information** to view a list of packages that are included in the image that you selected.

If the workbench image has multiple versions available, select the version to use from the **Version selection** list. Red Hat recommends that you use the latest version.



NOTE

You can change the workbench image after you create the workbench.

5. In the **Deployment size** section, from the **Hardware profile** list, select a suitable hardware profile for your workbench.

If project-scoped hardware profiles exist, the **Hardware profile** list includes subheadings to distinguish between global hardware profiles and project-scoped hardware profiles.

The hardware profile specifies the CPU and memory requests and limits for the container.

- a. Optional: To change the default values, click **Customize resource requests and limits**
6. Optional: In the **Environment variables** section, add environment variables to provide credentials or configuration values to the workbench. Setting environment variables during workbench configuration means you do not need to define them in the body of your notebooks or with the IDE command line interface.
Environment variables set here are available in the workbench container when it starts.

- a. Click **Add environment variable**.
- b. From the **Variable type** list, select the type of environment variable to add:
 - Select **Secret** to create a new Kubernetes Secret with key-value pairs entered directly or uploaded from a file.
 - Select **Config Map** to create a new ConfigMap for non-sensitive configuration values.
 - Select **Existing secret** to reference a pre-existing Kubernetes Secret already present in the project namespace. Use this option for credentials managed outside OpenShift AI by your platform team or external tools.
- c. If you selected **Secret**, choose a data entry method:
 - Select **Key / value** to enter key-value pairs manually.
 - Select **Upload** to import key-value pairs from an environment file.
- d. If you selected **Existing secret**, complete the following steps:
 - i. From the **Secrets** dropdown, search for and select one or more secrets by name. Only Kubernetes Secrets of type **Opaque** that are not managed by the connections framework appear in this dropdown.
 - ii. For each selected secret, expand the secret entry to view its keys. Choose **Select all** to inject all keys, or select individual key checkboxes to inject specific keys. A badge displays the number of selected keys out of the total keys available.
 - iii. Required: Review any environment variable name conflict warnings.
If the same key name exists in multiple sources, such as across existing secrets and inline secrets, the form displays a warning that lists the conflicting key names and their sources. To resolve a conflict, clear the conflicting key from one source, remove a secret reference, or rename an inline key.



NOTE

Environment variables from existing secrets are set at workbench startup. If secret values change after the workbench starts, for example during a credential rotation, restart the workbench to pick up the new values.

- e. Optional: To add another environment variable, click **Add environment variable** and repeat the preceding steps.

For more information about environment variable types and secret eligibility, see [Environment variable types for workbenches](#). For detailed reference information about the existing secret interface, see [Existing secret reference details for workbenches](#).

7. In the **Cluster storage** section, configure the storage for your workbench. Select one of the following options:

- **Create new persistent storage** to create storage that is retained after you shut down your workbench. Complete the relevant fields to define the storage:
 - a. Enter a **name** for the cluster storage.
 - b. Enter a **description** for the cluster storage.
 - c. Select a **storage class** for the cluster storage.



NOTE

You cannot change the storage class after you add the cluster storage to the workbench.

- d. For storage classes that support multiple access modes, select an **Access mode** to define how the volume can be accessed. For more information, see [About persistent storage](#).
Only the access modes that have been enabled for the storage class by your cluster and OpenShift AI administrators are visible.
 - e. Under **Persistent storage size**, enter a new size in gibibytes or mebibytes.
 - **Use existing persistent storage** to reuse existing storage and select the storage from the **Persistent storage** list.
8. Optional: You can add a connection to your workbench. A connection is a resource that contains the configuration parameters needed to connect to a data source or sink, such as an object storage bucket. You can use storage buckets for storing data, models, and pipeline artifacts. You can also use a connection to specify the location of a model that you want to deploy. In the **Connections** section, use an existing connection or create a new connection:
 - Use an existing connection as follows:
 - a. Click **Attach existing connections**.
 - b. From the **Connection** list, select a connection that you previously defined.
 - Create a new connection as follows:
 - a. Click **Create connection**. The **Add connection** dialog opens.
 - b. From the **Connection type** drop-down list, select the type of connection. The **Connection details** section is displayed.
 - c. If you selected **S3 compatible object storage** in the preceding step, configure the connection details:
 - i. In the **Connection name** field, enter a unique name for the connection.
 - ii. Optional: In the **Description** field, enter a description for the connection.

- iii. In the **Access key** field, enter the access key ID for the S3-compatible object storage provider.
 - iv. In the **Secret key** field, enter the secret access key for the S3-compatible object storage account that you specified.
 - v. In the **Endpoint** field, enter the endpoint of your S3-compatible object storage bucket.
 - vi. Optional: In the **Region** field, enter the default region of your S3-compatible object storage account.
 - vii. Optional: In the **Bucket** field, enter the name of your S3-compatible object storage bucket.
 - viii. Click **Create**.
- d. If you selected **URI** in the preceding step, configure the connection details:
- i. In the **Connection name** field, enter a unique name for the connection.
 - ii. Optional: In the **Description** field, enter a description for the connection.
 - iii. In the **URI** field, enter the Uniform Resource Identifier (URI).
 - iv. Click **Create**.
9. Click **Create workbench**.

Verification

- The workbench that you created is visible on the **Workbenches** tab for the project.
- Any cluster storage that you associated with the workbench during the creation process is displayed on the **Cluster storage** tab for the project.
- The **Status** column on the **Workbenches** tab displays a status of **Starting** when the workbench server is starting, and **Ready** when the workbench has successfully started.
- Optional: Click the open icon () to open the IDE in a new window.

Additional resources

- [Working with data in an S3-compatible object store](#)
- [About persistent storage](#)

CHAPTER 5. NEXT STEPS

The following product documentation provides more information on how to develop, test, and deploy data science solutions with OpenShift AI.

Try the end-to-end tutorial

[OpenShift AI tutorial - Fraud detection example](#)

Step-by-step guidance to complete the following tasks with an example fraud detection model:

- Explore a pre-trained fraud detection model by using a Jupyter notebook.
- Deploy the model by using OpenShift AI model serving.
- Refine and train the model by using automated pipelines.

Develop and train a model in your workbench IDE

[Working in your data science IDE](#)

Learn how to access your workbench IDE (JupyterLab, code-server, or RStudio Server).

For the JupyterLab IDE, learn about the following tasks:

- Creating and importing Jupyter notebooks
- Using Git to collaborate on Jupyter notebooks
- Viewing and installing Python packages
- Troubleshooting common problems

Automate your ML workflow with pipelines

[Working with AI pipelines](#)

Enhance your projects on OpenShift AI by building portable machine learning (ML) workflows with AI pipelines, by using Docker containers. Use pipelines for continuous retraining and updating of a model based on newly received data.

Deploy and test a model

[Deploying models](#)

Deploy your ML models on your OpenShift cluster to test and then integrate them into intelligent applications. When you deploy a model, it is available as a service that you can access by using API calls. You can return predictions based on data inputs that you provide through API calls.

Monitor and manage models

[Deploying models](#)

The Red Hat OpenShift AI service includes model deployment options for hosting the model on Red Hat OpenShift Dedicated or Red Hat OpenShift Service on AWS for integration into an external application.

Add accelerators to optimize performance

[Working with accelerators](#)

If you work with large data sets, you can use accelerators, such as NVIDIA GPUs, AMD GPUs, and Intel Gaudi AI accelerators, to optimize the performance of your data science models in OpenShift AI. With accelerators, you can scale your work, reduce latency, and increase productivity.

Implement distributed workloads for higher performance

[Working with distributed workloads](#)

Implement distributed workloads to use multiple cluster nodes in parallel for faster, more efficient data processing and model training.

Explore extensions

[Working with connected applications](#)

Extend your core OpenShift AI solution with integrated third-party applications. Several leading AI/ML software technology partners, including Starburst, Intel AI Tools, and IBM are also available through [Red Hat partners](#) and [IBM Partner Plus Directory](#).

5.1. ADDITIONAL RESOURCES

In addition to product documentation, Red Hat provides a rich set of learning resources for OpenShift AI and supported applications.

On the **Resources** page of the OpenShift AI dashboard, you can use the category links to filter the resources for various stages of your data science workflow. For example, click the **Model serving** category to display resources that describe various methods of deploying models. Click **All items** to show the resources for all categories.

For the selected category, you can apply additional options to filter the available resources. For example, you can filter by type, such as how-to articles, quick starts, or tutorials; these resources provide the answers to common questions.

For information about Red Hat OpenShift AI support requirements and limitations, see [Supported Configurations for 3.x](#).